

Práctica Java 1 de Septiembre

Juan Guillermo Flórez G.

Tengan cuidado con la IA. Si no se sabe sustentar su calificación será 0.

Sistema de análisis y control de producción industrial

Una empresa manufacturera dedicada a la producción de piezas metálicas para el sector automotriz necesita desarrollar un módulo en Java que permita analizar la producción diaria de sus diferentes líneas de fabricación.

Cada registro de producción debe contener información como:

- código del producto
- nombre
- línea de producción
- cantidad producida
- cantidad defectuosa
- costo unitario
- minutos utilizados
- kilogramos de materia prima consumidos y meta de producción.

La empresa requiere:

- identificar productos con niveles altos de defectos
- calcular el cumplimiento de metas
- conocer el costo total de fabricación

- determinar pérdidas económicas asociadas a productos defectuosos
- analizar el desempeño general de cada línea de producción.

El sistema debe permitir transformar los registros de producción en información resumida para los supervisores

- obtener los productos de mayor y menor desempeño
- identificar líneas con bajo cumplimiento
- calcular la cantidad total producida
- determinar cuánto dinero se ha invertido en producción.

También será necesario generar automáticamente:

- un registro de producción de prueba
- modificar cantidades producidas cuando se reporten unidades adicionales
- aplicar ajustes porcentuales sobre determinados registros
- ejecutar un proceso de cierre del turno.

Al finalizar, el sistema deberá mostrar un reporte con:

- las líneas existentes
- producción total por línea
- productos críticos
- productos que superaron la meta
- unidades defectuosas
- costo total de producción
- pérdidas económicas

- producto con mayor pérdida
- línea con mayor cantidad producida.

La solución deberá desarrollarse principalmente utilizando programación funcional en Java, evitando ciclos tradicionales para las operaciones de análisis y transformación cuando puedan resolverse utilizando Streams.

Funciones e interfaces que se pueden utilizar Expresiones Lambda
 Stream Predicate Function Consumer Supplier BiFunction BiPredicate
 BiConsumer UnaryOperator BinaryOperator Runnable Callable stream()
 filter() map() mapToInt() mapToDouble() forEach() reduce() collect()
 sorted() distinct() limit() skip() count() max() min() findFirst() findAny()
 anyMatch() allMatch() noneMatch() Collectors.toList() Collectors.toSet()
 Collectors.groupingBy() Collectors.partitioningBy()
 Collectors.summingInt() Collectors.summingDouble()
 Collectors.averagingDouble() Collectors.counting() Collectors.mapping()
 Collectors.joining() Comparator.comparing()
 Comparator.comparingDouble() reversed()